```
#Mount the Google Drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
#unpack the Zip file
import shutil
shutil.unpack_archive('/content/drive/MyDrive/Colab Notebooks/Datasets/Capstone1_Part1/Images.zip')
print("UnZipped")
```

```
UnZipped
```

```
#Images are extracted in the Content Images Folder
images_dir='content/Images'
```

```
import pandas as pd
import os
import cv2
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

print("Imported Libraries")
```

```
Imported Libraries
```

```
# Load the labels from labels.csv
labels_df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Datasets/Capstone1_Part1/labels.csv', sep=',', header=None)
labels_df.columns = ['image_id', 'class', 'x_min', 'y_min', 'x_max', 'y_max']
print("Imported the Labels Dataset")
```

```
Imported the Labels Dataset
```

```
labels_df
```

	image_id	class	x_min	y_min	x_max	y_max
0	0	pickup_truck	213	34	255	50
1	0	car	194	78	273	122
2	0	car	155	27	183	35
3	0	articulated_truck	43	25	109	55
4	0	car	106	32	124	45
...	...	...	...	...	...	...
351544	110590	car	18	57	97	98
351545	110591	articulated_truck	2	71	690	351
351546	110592	pickup_truck	3	240	214	378
351547	110592	car	465	111	507	135
351548	110592	non-motorized_vehicle	197	187	318	269

351549 rows × 6 columns

we cannot proceed with large dataset since the GPU / Compute limitations in the Google Colab, so create a demo model with less number of the images. so implement the model with small number of images

Double-click (or enter) to edit

```
#Do zero padding for the image id to match the image name in the google drive and we are taking only 1000 images

labels_df['image_id'] = labels_df['image_id'].apply(lambda x: f"{x:08d}")
labels_df
```

	image_id	class	x_min	y_min	x_max	y_max
0	00000000	pickup_truck	213	34	255	50
1	00000000	car	194	78	273	122
2	00000000	car	155	27	183	35
3	00000000	articulated_truck	43	25	109	55
4	00000000	car	106	32	124	45
...	...	...	...	...	...	...
351544	00110590	car	18	57	97	98
351545	00110591	articulated_truck	2	71	690	351
351546	00110592	pickup_truck	3	240	214	378
351547	00110592	car	465	111	507	135
351548	00110592	non-motorized_vehicle	197	187	318	269

351549 rows × 6 columns

```
# Use iloc to pick the first 1000 labels
labels_df = labels_df.iloc[:1000]
labels_df
```

	image_id	class	x_min	y_min	x_max	y_max
0	00000000	pickup_truck	213	34	255	50
1	00000000	car	194	78	273	122
2	00000000	car	155	27	183	35
3	00000000	articulated_truck	43	25	109	55
4	00000000	car	106	32	124	45
...	...	...	...	...	...	...
995	00000306	motorized_vehicle	36	44	55	56
996	00000306	car	110	93	208	140
997	00000306	articulated_truck	55	19	111	37
998	00000307	car	2	233	17	283
999	00000307	work_van	75	46	115	73

1000 rows × 6 columns

image data pre processing, reshaping, resizing

```
images_dir = 'Images/'
images = []
target_size = (224, 224)  # Adjust this size according to your model's requirements.
for index, row in labels_df.iterrows():
    img_path = os.path.join(images_dir, f"{row['image_id']}.jpg")
    img = cv2.imread(img_path)
    if img is not None:
        if len(img.shape) == 2:  # If grayscale, convert to RGB
            img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
        img = cv2.resize(img, target_size)  # Resize image
        images.append(img)
    else:
        print(f"Error loading image: {img_path}")
images = np.array(images)  # Convert list of images to a NumPy array
if len(images) == 0:
    print("No images loaded. Please check the image paths.")
```

```
else:
    print(f"{len(images)} images loaded successfully.")
```

```
1000 images loaded successfully.
```

```
print("Total Images :",len(images)) #1000
print("Image Size : ", images[0].shape) #224*224
```

```
Total Images : 1000
Image Size :  (224, 224, 3)
```

```
# Analyze the distribution of vehicle types in the limited dataset
vehicle_types = labels_df['class'].value_counts()
print("Distribution of vehicle types:")
print(vehicle_types)
```

```
Distribution of vehicle types:
class
car                     682
pickup_truck            111
motorized_vehicle        61
articulated_truck        30
work_van                 29
bus                      28
pedestrian               23
single_unit_truck        18
bicycle                  12
non-motorized_vehicle     5
motorcycle                1
Name: count, dtype: int64
```

```
# Address data quality issues arising from the discrepancy between labels and actual image filenames
# Sorting the image filenames
labels_df = labels_df.sort_values('image_id')
print("Sorted")
```

```
Sorted
```

```
#Image Quality Check and resize to 224*224

if len(images) > 0:
    processed_images = [cv2.resize(img, (224, 224)) for img in images]  # Adjust dimensions as needed
    processed_images = np.array(processed_images)
    print("Images resized successfully.")
```

```
Images resized successfully.
```

```
#Data Preprocessing for Labels
labels = labels_df['class'].to_numpy()
bounding_boxes = labels_df[['x_min', 'y_min', 'x_max', 'y_max']].to_numpy()
```

```
#Convert labesl to one-hot encoding, labels are converted into integer numbers, because models needs integer values
unique_labels = np.unique(labels)
label_to_index = {label: index for index, label in enumerate(unique_labels)}
index_to_label = {index: label for index, label in enumerate(unique_labels)}
labels = np.array([label_to_index[label] for label in labels])
labels
```

```
array([ 8,  3,  3,  0,  3,  2,  2,  3,  3,  3,  3,  5,  3,  3,  2,  3,  0,
        3,  3,  8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,
        8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  2,  3,  3,  8,  3,  0, 10,
        5,  3,  5,  3,  3,  3,  3,  3,  0,  3,  3,  3,  2,  2,  3,  5,  3,
        3,  3,  3,  5,  8,  3,  8,  3,  3,  8,  3,  3,  3,  3,  9,  9,  3,
        8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  0,  3,  3,  3,  8,  3,
        3,  9,  3,  3,  8,  3,  8,  3,  3,  8,  3,  3,  3,  3,  3,  3,  3,
        3,  3,  3,  3, 10,  2,  3,  3,  3,  5, 10,  3,  3,  3,  3,  3,  3,
        8,  3,  3,  3,  3,  3,  3,  3,  5,  7,  3,  3,  5,  3,  3,  3,  8,
        3,  3,  8,  3,  3,  3,  5,  3,  8,  8, 10,  8,  8,  3,  8,  3,  3,
        3,  5,  3,  5,  3,  5,  8,  8,  8,  3,  3, 10,  0,  5,  3,  8,  3,
        3,  3,  8,  8,  5,  3,  3,  3,  3,  3,  8,  8,  3,  8, 10,  3,  3,
        8,  3,  8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  5,  0,  3,  3,  3,
        3,  3,  1,  3,  3,  3,  3,  3,  3,  8,  3,  3,  3,  3,  3,  3,  3,
        3,  3, 10,  3,  3,  3,  3,  5,  3,  3, 10,  3,  3,  3,  3,  3, 10,
        3,  3,  3,  5,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,
        3,  3,  5,  3,  3,  3,  3,  3,  5,  3,  3,  3,  3,  3,  3,  3,  3,
        2,  3,  3, 10,  3,  3,  3,  3,  3,  3,  3,  3,  0,  2,  3,  0,  3,
        3,  8,  8,  3,  3,  3,  8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  2,
```

```
 3,  3,  3,  8,  2,  3,  3,  9,  3,  3,  8,  3,  3,  3,  3,  3,  3,
 3,  3,  3,  3,  3,  3,  3,  3,  0,  0,  3,  3,  3,  3,  3,  3,  3,
 2,  3,  3,  3,  8,  7,  1,  7,  7,  1, 10,  3,  3,  3,  3,  3,  3,
 3,  3,  3,  5,  7,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,
 3,  3,  3,  2,  3,  3,  3,  3,  3,  3,  3,  8,  3,  8,  5,  5,  3,
10,  3,  5,  3, 10,  3,  3,  8,  3,  3,  6,  3,  3,  3,  3,  3,  3,
 3,  3, 10,  8,  5,  8,  8,  5,  3, 10,  3,  3,  8,  8,  5,  3,  3,
 3,  2,  5,  8,  5,  8,  3,  8,  3,  5,  3,  8,  3,  3,  3,  0,  3,
10,  9,  3,  3,  6,  3,  3,  3,  3,  3,  3,  3,  3,  1,  1,  3,  3,
 3,  3,  3,  3,  3,  3,  0,  3,  7,  3,  7,  3,  3,  3,  3,  3,  3,
 5,  3,  0,  9,  5, 10,  3,  8,  3,  2,  3,  5,  3,  3,  3,  3,  3,
 3,  3,  3,  3,  3,  3,  8,  3,  3,  7,  3,  3,  3,  3,  3,  3,  3,
 3,  9,  3,  3,  3,  2,  3,  3,  3,  3,  6,  9,  6,  3,  3,  5,  8,
 8,  8,  3,  3,  0,  3,  3,  3,  3,  3,  3,  8,  3,  8,  3,  3,  3,
 3,  3,  5,  3,  3,  0,  3,  8,  8,  3,  3,  3,  3,  3,  2,  3,  0,
 8,  3,  3,  0,  3,  8,  3,  3,  3,  3,  3,  3,  3,  3,  3,  1,  1,
 1,  3,  8,  8,  3,  3,  8,  3,  9,  3,  3,  3,  8,  3,  3,  3,  3,
 3,  3,  3,  3,  0, 10, 10,  2,  3,  3,  8,  3,  3,  8,  8,  5,  3,
 3,  3,  0,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  8,  3,  3,  8,
 8,  3, 10,  8,  5,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,
 3,  3,  7,  3,  3,  3,  3,  3,  3,  3,  3,  3,  7,  7,  3,  5,  3,
 3,  7,  3,  5,  5,  5,  3,  3,  3,  3,  3,  8,  8,  3, 10,  3,  3,
 8,  3,  3,  8,  9,  6,  3,  8,  3,  3,  3,  3,  5,  9,  3,  2,  3,
 7, 10,  3,  3,  3,  3,  3,  5,  7,  3,  7,  8,  3,  8,  3,  0,  3,
 9,  0,  0,  5, 10,  0,  3,  3,  3,  3,  8,  3,  3,  3,  3,  3,  3,
 3,  3,  7,  7,  3,  3,  2,  3,  3,  8,  3,  3,  5,  3,  3,  3,  3,
 3,  2,  3,  8,  3,  3,  3,  3,  5,  3,  3,  3,  3,  9,  3,  3,  8,
 0,  3,  5,  3,  5,  2,  3,  3,  8,  2,  5,  3,  0,  5,  8,  3,  3,
 3, 10,  3,  3,  3,  3,  2,  3,  2, 10,  3,  3,  3,  3,  2,  3,  3,
 3,  3,  3,  3,  3,  3,  3,  5,  3,  3,  3,  3,  7,  7,  3,  3,  3,
 7,  3,  3,  3,  3,  4,  3,  3,  3,  3,  3,  3,  8,  3,  3,  3,  3,
 3,  8,  5,  3,  3,  8,  3,  2,  8,  8,  2,  3,  3,  9,  3,  3,  3,
 8,  3,  3,  1,  3,  3,  8,  3,  0,  3,  3,  9, 10,  5,  3,  3,  3,
 3,  3,  3,  3,  5,  5,  5,  3,  3,  8,  3,  3,  9,  3,  8,  0,  3,
 3,  9,  3,  3,  3,  3,  8,  3,  3,  5,  3,  8,  5,  3,  3,  8,  8,
 8,  9,  3,  8,  3,  1,  3,  1,  3,  8,  3,  5,  8,  3,  8,  3,  3,
 8,  3,  3,  3,  3,  7, 10,  3,  3,  3,  3,  3,  3,  3,  7,  1,  3,
 3,  3,  3,  8,  8,  3,  3,  3,  3,  3,  3,  3, 10,  8,  7,  3,  3,
```

Model Building

```
#Train the Images with 80% vs 20% with 80% as training data
X_train, X_test, y_train, y_test, bbox_train, bbox_test = train_test_split(processed_images, labels, bounding_boxes, test_size=
```

```
print("X_train Shape:", X_train.shape)
print("y_train Shape:", y_train.shape)
print("bbox_train shape:", bbox_train.shape)
print("First few y_train labels:", y_train[:5])
print("First few bbox_train labels:", bbox_train[:5])
```

```
X_train Shape: (800, 224, 224, 3)
y_train Shape: (800,)
bbox_train shape: (800, 4)
First few y_train labels: [3 3 3 8 3]
First few bbox_train labels: [[476 148 610 206]
 [573 219 675 285]
 [295 121 345 142]
 [324 110 381 128]
 [380 192 462 221]]
```

Create an CNN Architecture for Object Detection

```
#we have 2 output layers
def create_model(input_shape, num_classes):
    inputs = keras.Input(shape=input_shape)
    x = layers.Conv2D(32, (3, 3), activation='relu')(inputs)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Conv2D(64, (3, 3), activation='relu')(x)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Conv2D(64, (3, 3), activation='relu')(x)
    x = layers.Flatten()(x)
    x = layers.Dense(64, activation='relu')(x)
    vehicle_class = layers.Dense(num_classes, activation='softmax', name='vehicle_class')(x)
    bounding_box = layers.Dense(4, name='bounding_box')(x)
```

```
    model = keras.Model(inputs=inputs, outputs=[vehicle_class, bounding_box])
    return model
```

```
#Call the Model

input_shape = processed_images[0].shape
num_classes = len(unique_labels)
model = create_model(input_shape, num_classes)
```

```
#Compile the Model.

model.compile(optimizer='adam',
              loss={'vehicle_class': 'sparse_categorical_crossentropy', 'bounding_box': 'mse'},
              metrics={'vehicle_class': 'accuracy', 'bounding_box': 'mae'})
```

```
#Run the Model
model.fit(X_train, {'vehicle_class': y_train, 'bounding_box': bbox_train}, epochs=50, validation_data=(X_test, {'vehicle_class'
```

```
Epoch 1/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 12s 158ms/step - bounding_box_loss: 329841.4688 - bounding_box_mae: 273.9555 - loss: 330361.7500 - v
Epoch 2/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - bounding_box_loss: 24485.3047 - bounding_box_mae: 123.8987 - loss: 25115.7676 - vehi
Epoch 3/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 51ms/step - bounding_box_loss: 20540.7969 - bounding_box_mae: 110.9438 - loss: 20836.0547 - vehi
Epoch 4/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - bounding_box_loss: 19292.6484 - bounding_box_mae: 107.3581 - loss: 19367.3203 - vehi
Epoch 5/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 48ms/step - bounding_box_loss: 18995.7480 - bounding_box_mae: 105.2281 - loss: 19032.1641 - vehi
Epoch 6/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - bounding_box_loss: 18570.8438 - bounding_box_mae: 103.2755 - loss: 18601.4062 - vehi
Epoch 7/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 18010.5645 - bounding_box_mae: 102.9671 - loss: 18039.9141 - vehi
Epoch 8/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 17719.9141 - bounding_box_mae: 101.2109 - loss: 17751.8301 - vehi
Epoch 9/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - bounding_box_loss: 16810.2520 - bounding_box_mae: 97.7290 - loss: 16837.7754 - vehic
Epoch 10/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - bounding_box_loss: 16617.4414 - bounding_box_mae: 97.4055 - loss: 16643.3984 - vehic
Epoch 11/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 16045.9053 - bounding_box_mae: 94.7027 - loss: 16071.1064 - vehic
Epoch 12/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 16155.9805 - bounding_box_mae: 95.0765 - loss: 16180.8428 - vehic
Epoch 13/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - bounding_box_loss: 16249.9775 - bounding_box_mae: 95.3320 - loss: 16276.4053 - vehic
Epoch 14/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 52ms/step - bounding_box_loss: 16199.7900 - bounding_box_mae: 94.9069 - loss: 16222.5078 - vehic
Epoch 15/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - bounding_box_loss: 16452.3770 - bounding_box_mae: 95.2842 - loss: 16475.0254 - vehic
Epoch 16/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - bounding_box_loss: 16398.2441 - bounding_box_mae: 95.0007 - loss: 16419.9531 - vehic
Epoch 17/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 50ms/step - bounding_box_loss: 15683.4375 - bounding_box_mae: 92.7630 - loss: 15703.7461 - vehic
Epoch 18/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 15504.4287 - bounding_box_mae: 90.8572 - loss: 15524.8760 - vehic
Epoch 19/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 15461.0986 - bounding_box_mae: 92.4691 - loss: 15483.4717 - vehic
Epoch 20/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 15804.9922 - bounding_box_mae: 92.6426 - loss: 15824.4033 - vehic
Epoch 21/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 16045.5820 - bounding_box_mae: 93.0473 - loss: 16064.8350 - vehic
Epoch 22/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 14996.4424 - bounding_box_mae: 89.3004 - loss: 15017.4199 - vehic
Epoch 23/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 49ms/step - bounding_box_loss: 15000.7324 - bounding_box_mae: 90.4837 - loss: 15020.7510 - vehic
Epoch 24/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - bounding_box_loss: 15084.8750 - bounding_box_mae: 90.4616 - loss: 15102.9023 - vehic
Epoch 25/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 54ms/step - bounding_box_loss: 15314.7734 - bounding_box_mae: 89.9658 - loss: 15332.7461 - vehic
Epoch 26/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 50ms/step - bounding_box_loss: 15607.8203 - bounding_box_mae: 91.1990 - loss: 15628.0498 - vehic
Epoch 27/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 53ms/step - bounding_box_loss: 14917.7227 - bounding_box_mae: 89.4703 - loss: 14934.1387 - vehic
Epoch 28/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 2s 66ms/step - bounding_box_loss: 14694.0010 - bounding_box_mae: 88.6771 - loss: 14710.5811 - vehic
Epoch 29/50
```

```
#Evaulate the model. Error will be higher, the reason the poor model architecture and less number of images
test_results = model.evaluate(X_test, {'vehicle_class': y_test, 'bounding_box': bbox_test}, verbose=2)
print('\nTest results:', test_results)
```

```
7/7 - 0s - 45ms/step - bounding_box_loss: 25182.0176 - bounding_box_mae: 114.6976 - loss: 24093.3809 - vehicle_class_accuracy: 0

Test results: [24093.380859375, 10.456997871398926, 25182.017578125, 114.69761657714844, 0.6050000190734863]
```

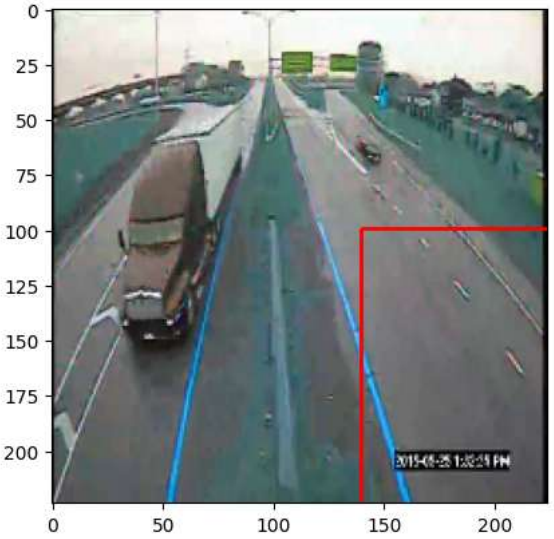
Test the model by passing actual images

```
import matplotlib.pyplot as plt
# Choose a few sample images for inference
sample_images = X_test[:20]  # Adjust the number of sample images as needed
# Perform inference on the sample images
predictions = model.predict(sample_images)
# Extract the predicted bounding box coordinates
predicted_bounding_boxes = predictions[1]
# add labels also in for loop
# Extract the predicted labels
predicted_labels = [index_to_label[label] for label in np.argmax(predictions[0], axis=1)]
# Visualize the sample images with predicted bounding boxes
for i in range(len(sample_images)):
    plt.figure()
    plt.imshow(sample_images[i])
    # add labels also
    plt.title(f"Predicted Label: {predicted_labels[i]}")
    plt.gca().add_patch(plt.Rectangle((predicted_bounding_boxes[i][0], predicted_bounding_boxes[i][1]),
                                      predicted_bounding_boxes[i][2] - predicted_bounding_boxes[i][0],
                                      predicted_bounding_boxes[i][3] - predicted_bounding_boxes[i][1],
                                      fill=False, edgecolor='r', linewidth=2))
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

Predicted Label: 3

Predicted Label: 3

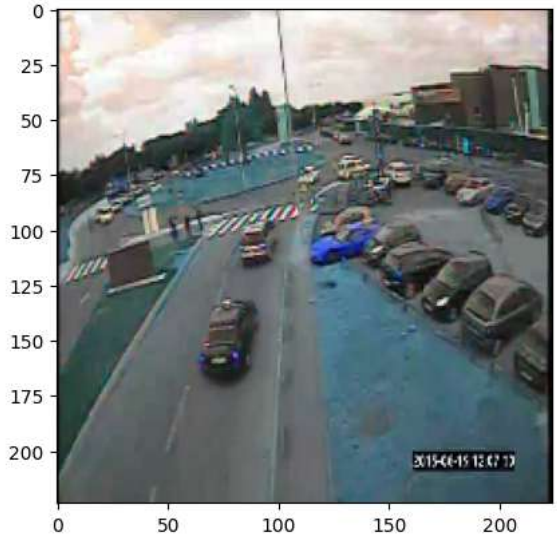


Predicted Label: 8

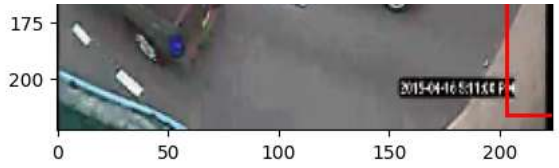
Predicted Label: 3

Predicted Label: 8

Predicted Label: 3

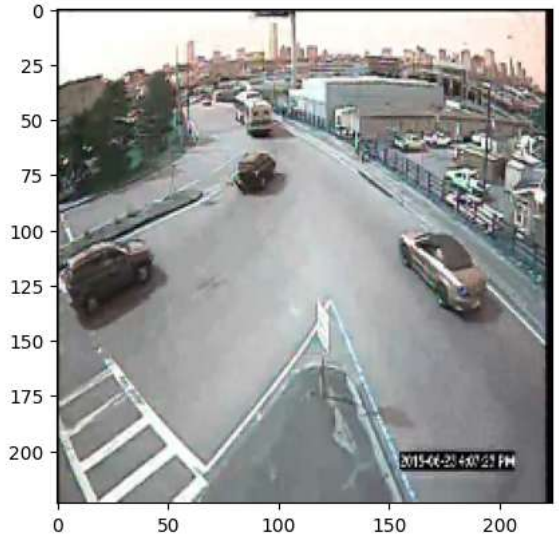


Predicted Label: 3



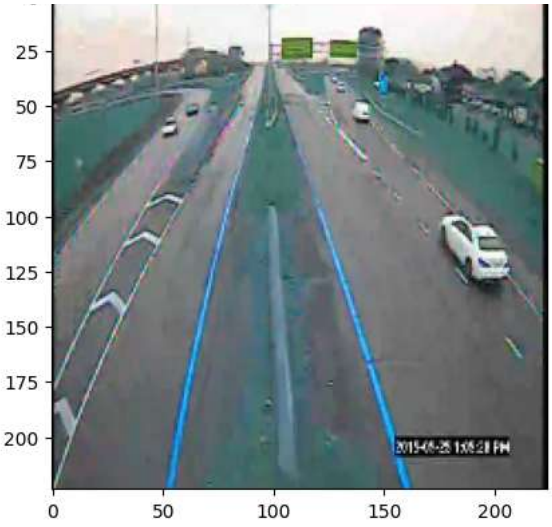
Predicted Label: 3

Predicted Label: 8



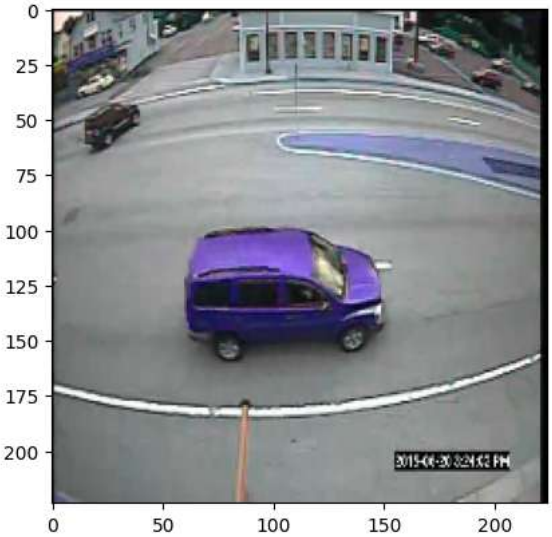
Predicted Label: 3

Predicted Label: 3



Predicted Label: 5

Predicted Label: 3



Predicted Label: 3

Predicted Label: 3

Predicted Label: 3